

NEXUS

NEEDS EXCHANGE & UNIFIED COORDINATION SYSTEM

Frontend Architecture & Requirements Document

Comprehensive Technical Specification for Stakeholders

NEXT.JS 14

TYPESCRIPT

REACT QUERY

MAPBOX GL

SOCKET.IO

PWA-READY

DOCUMENT SECTIONS

- 1 Project Setup & Dependency Matrix
- 2 Directory Structure
- 3 Routing & Page Architecture
- 4 API Mapping & Gap Analysis
- 5 State Management Strategy

Document Context

This specification is derived directly from the NEXUS backend built across Phases 0–5 of this engagement — a multi-tenant, cloud-native volunteer coordination platform for NGOs in India. Every API endpoint, data model, user role, and business rule referenced herein has a corresponding backend implementation. The Gap Analysis in Section 4 identifies the delta that requires backend additions before frontend development can begin.

Backend Services	Auth · Registry · Ingestion · Intelligence · Coordination · Analytics
User Roles	field_worker · coordinator · ngo_admin · funder_readonly · platform_admin
Languages	English · Hindi · Marathi · Tamil (all UI + API templates)
Real-time	Socket.io WebSocket on Node.js + 10 Kafka event types
Database	PostgreSQL + PostGIS · MongoDB · Elasticsearch · Redis
API Endpoints	70+ REST endpoints across 6 microservices + WebSocket
Deployment Target	AWS + Kubernetes · India region (ap-south-1)

SECTION 1 — PROJECT SETUP & DEPENDENCY MATRIX

1.1 Framework Decision — Next.js 14 (App Router) + TypeScript

Next.js 14 with the App Router is the optimal choice given four non-negotiable requirements: (1) SSR for funder report pages which must be indexable and shareable without authentication, (2) middleware-level JWT validation and multi-tenant subdomain routing before any React renders, (3) API Route Handlers that proxy to backend microservices hiding internal URLs from clients, and (4) seamless integration with the Node.js Socket.io WebSocket layer already built in the coordination service.

Criterion	Next.js 14 ■	Vite + React	Remix
SSR for funder reports	■ Native (ISR)	■ Manual	■ Native
JWT middleware routing	■ next/middleware	■■ Manual	■ Loaders
API proxy layer	■ Route Handlers	■ None	■ Loaders
Mapbox GL JS (heatmap)	■ Dynamic import	■ Direct	■ Direct
Multi-tenant subdomain	■ Middleware	■■ Manual	■■ Manual
WebSocket + Socket.io	■ Custom server	■ Direct	■■ Partial
PWA / Service Worker	■ next-pwa plugin	■ Vite PWA	■■ Partial

1.2 Scaffolding

TERMINAL

```
npx create-next-app@latest nexus-web \
  --typescript --tailwind --eslint --app \
  --src-dir --import-alias '@/*'
cd nexus-web
cp .env.example .env.local
```

1.3 Complete Dependency Matrix

Core Framework		
Package	Version	Justification
next	^14.2.x	App Router, Middleware, Route Handlers, ISR
react / react-dom	^18.3.x	React 18 concurrent features
typescript	^5.4.x	Type safety across all 70+ API contracts
Routing & Internationalisation		
Package	Version	Justification
next-intl	^3.x	i18n routing for EN/HI/MR/TA — mirrors backend preferred_language field
State Management		
Package	Version	Justification
zustand	^4.5.x	Global auth/tenant/socket state — lightweight, no boilerplate
@tanstack/react-query	^5.x	Server state: caching, background refetch, optimistic updates
immer	^10.x	Immutable Zustand state updates

UI & Styling

Package	Version	Justification
tailwindcss	^3.4.x	Utility-first, maps cleanly to NEXUS design tokens
@radix-ui/react-*	^1.x	Accessible primitives: Dialog (merge confirm), Select (filter), Tooltip (score)
lucide-react	^0.383.x	Icon library — consistent with existing NEXUS artifact work
clsx + tailwind-merge	^2.x	Conditional class composition without conflicts
class-variance-authority	^0.7.x	Component variant system for shared UI elements

Geospatial

Package	Version	Justification
mapbox-gl	^3.x	Powers the urgency geo-heatmap (geohash_grid aggregation from Elasticsearch)
react-map-gl	^7.x	React wrapper for Mapbox GL JS
@turf/turf	^6.x	Client-side GeoJSON: ward boundaries, volunteer proximity display

Data Visualization

Package	Version	Justification
recharts	^2.12.x	Impact metrics line charts, category breakdown, HII trend charts
@visx/heatmap	^3.x	Ward urgency heatmap grid (non-geo tabular complement to Mapbox)

Forms & Validation

Package	Version	Justification
react-hook-form	^7.51.x	Field worker mobile form builder — matches MobileFormSubmission Pydantic schema
zod	^3.23.x	Schema validation mirroring Pydantic models from backend
@hookform/resolvers	^3.x	Bridges Zod schemas into react-hook-form

Network & Real-time

Package	Version	Justification
axios	^1.7.x	HTTP client with interceptors: JWT refresh, tenant header, correlation-ID injection
socket.io-client	^4.7.x	Connects to the Node.js + Socket.io WebSocket layer we built

File & Media

Package	Version	Justification
react-dropzone	^14.x	Paper survey image upload → POST /ingest/image
browser-image-compression	^2.x	Compress photos before upload (rural bandwidth constraint)
papaparse	^5.x	Client-side CSV preview/validation before POST /ingest/csv

Utilities

Package	Version	Justification
date-fns	^3.x	Date formatting for ingested_at, reported_at, urgency time-decay display
uuid	^9.x	Client-side correlation_id generation before API calls
js-cookie	^3.x	Secure refresh token management (httpOnly via proxy)
@sentry/nextjs	^8.x	Error monitoring + distributed tracing

Dev Tools		
Package	Version	Justification
msw	^2.x	Mock Service Worker — mock all 70+ backend endpoints during development
@testing-library/react	^15.x	Component testing
vitest	^1.x	Fast unit test runner
storybook	^8.x	Isolated component development for the shared design system

SECTION 2 — DIRECTORY STRUCTURE

2.1 Scalable Modular Folder Architecture

The structure follows Next.js App Router conventions with route groups isolating authentication contexts, role-based access, and the mobile PWA shell for field workers. Feature folders within ``components/`` and ``lib/`` mirror the backend service boundaries (registry, ingestion, intelligence, coordination, analytics), making API-to-component tracing straightforward.

DIRECTORY STRUCTURE

```
nexus-web/src/
├── app/
│   ├── (auth)/           # Unauthenticated: login, tenant login
│   ├── (platform)/       # platform_admin only: tenant management
│   └── (app)/             # Main authenticated shell
│       ├── layout.tsx    # AppShell + WebSocket init + role guard
│       ├── dashboard/    # Live coordinator dashboard (WebSocket)
│       ├── needs/        # Priority queue + review queue
│       ├── households/   # Registry: profile, consent, HII, merge
│       ├── volunteers/   # Registry: profile, burnout, performance
│       ├── tasks/        # Kanban board + dispatch controls
│       ├── ingest/       # 4 channels: mobile, image, text, CSV
│       ├── analytics/    # Impact metrics, HII table, overrides
│       ├── intelligence/ # Heatmap, priority queue, escalations
│       ├── settings/     # Weights, users, cross-tenant links
│       └── funder/        # funder_readonly isolated portal
│   ├── field/            # PWA shell for field workers (mobile)
│   └── api/proxy/         # Next.js route handlers (JWT proxy)
├── components/
│   ├── ui/               # Design system primitives (Radix + CVA)
│   ├── shared/           # UrgencyScoreBadge, BurnoutIndicator...
│   ├── dashboard/       # LiveEventFeed, EscalationAlert
│   ├── map/              # HeatmapLayer, VolunteerPins, WardLayer
│   ├── households/      # ConsentManager, HistoryTimeline
│   ├── ingestion/       # MobileNeedForm, ImageUploadZone, CSV
│   ├── tasks/           # MatchCandidateCard, DispatchControls
│   ├── analytics/       # ImpactMetricCard, HIITable, Charts
│   └── intelligence/    # WeightSliders, ScoreHistoryChart
├── hooks/                # useAuth, useSocket, useNeedsQuery...
├── lib/
│   ├── api/              # Axios client + per-service call wrappers
│   ├── schemas/          # Zod schemas mirroring Pydantic models
│   ├── stores/           # Zustand store slices
│   ├── constants/       # Roles, categories, task states
│   ├── utils/           # urgencyColor, roleGuard, formatDate
│   └── types/            # TypeScript interfaces for all API types
```

SECTION 3 — ROUTING & PAGE ARCHITECTURE

3.1 Route Guard Matrix

Route	Allowed Roles	Guard Mechanism
(auth)/*	Public (unauthenticated)	Middleware redirects authenticated users to /dashboard
(platform)/*	platform_admin only	Middleware validates JWT role claim before render
(app)/dashboard	coordinator, ngo_admin, platform_admin	Middleware + layout role check
(app)/needs/*	coordinator, ngo_admin, field_worker (read)	Per-page permission gate
(app)/households/*	coordinator, ngo_admin, field_worker (create)	Layout role check
(app)/volunteers/*	coordinator, ngo_admin	Layout role check
(app)/tasks/*	coordinator, ngo_admin	Layout role check
(app)/ingest/*	field_worker, coordinator, ngo_admin	Layout role check
(app)/analytics/*	coordinator, ngo_admin, funder_readonly	Funder sees anonymized subset only
(app)/intelligence/*	coordinator, ngo_admin	Layout role check
(app)/settings/*	ngo_admin, platform_admin	Layout role check
(app)/funder/*	funder_readonly	Middleware strict isolation — no raw data exposure
field/*	field_worker	Separate PWA shell, mobile-optimized

3.2 Key Page Specifications

(app)/dashboard — Live Coordinator Dashboard

- Components: LiveEventFeed, EscalationAlert[], WardStatsPanel, PriorityQueueTable (top 5), HeatmapLayer (mini-map), ResourceDesertAlert
- WebSocket events: need.created (toast + queue prepend), need.urgency_updated (in-place score update), task.assigned (Kanban move), volunteer.location_updated (pin update), analytics.gap_report.generated (desert alert)
- API calls: GET /priority-queue?limit=5, GET /escalations, GET /ward-stats, GET /gap-report

(app)/needs — Priority Queue (Full View)

- Components: PriorityQueueTable with UrgencyScoreBadge, CategoryBadge, FilterBar (category, ward, vulnerability)
- Clicking a row opens NeedCard drawer with MatchCandidateCard suggestions and DispatchControls
- Urgency score badge hover tooltip shows all 7 term values (t1–t7) from GET /score-history/{need_id}
- API calls: GET /priority-queue (paginated, filtered), GET /score-history/{need_id} (on hover)

(app)/needs/review — Human Review Queue

- Components: ReviewQueueTable with review_type badges (low_nlp_confidence / household_resolution), IdentityResolutionReview, LinkToHouseholdSearch
- Approving a household_resolution item renders side-by-side household comparison with 3 identity resolution candidates and confidence scores
- API calls: GET /review, POST /review/{id}/approve (with link_to_household), POST /review/{id}/reject

(app)/households/[id] — Household Profile (5 tabs)

- Profile: HouseholdProfile, VulnerabilityFlags (persistent, colour-coded), CrossTenantLinkBadge
- Members: MemberList — add/remove/mark-absent (soft delete via removed_at)
- Consent: ConsentManager — per consent type display with expiry countdown, inline revoke + full opt-out with DPDP Act 2023 disclaimer
- History: HistoryTimeline — chronological household_history ledger, filterable by event_type (13 enum values)
- Improvement: HIITable — 30/90/365-day windows with trend arrows (improving/stable/worsening) and crisis_frequency indicator

(app)/ingest/image — Paper Survey Upload

- Flow: Dropzone → client-side image compression → POST /ingest/image → OCR text in editable preview → user corrects → pipeline result shows need_id, household_status, review flag
- Components: ImageUploadZone, OCRPreview (editable), PipelineResult with household_status badge and review queue link
- Real constraint: OCR takes 3–15 seconds. Poll GET /ingest/status/{raw_id} at 2s intervals until status = processed

(app)/tasks/[id] — Task Detail & Dispatch

- Components: TaskStatusPill, StateTransitionLog (timeline), MatchCandidateCard[] (top 3), DispatchControls, OutcomeForm, BriefingPreview
- MatchCandidateCard shows: match_score (cosine + boosts), distance_km, burnout_risk (colour: green/amber/red), boosts_applied breakdown (continuity/language/cultural), estimated travel time
- Override path: if coordinator selects non-top candidate → OverrideReasonInput → POST /coordination/overrides/record THEN POST /tasks/{id}/dispatch
- BriefingPreview: POST /tasks/{id}/briefing/preview with volunteer_id and language → shows exact WhatsApp message before dispatch confirmed

(app)/intelligence/heatmap — Geo-Heatmap

- Components: HeatmapLayer (Mapbox geohash_grid → fill layer colour-scaled by avg_urgency), WardBoundaryLayer (polygon overlay), NeedMarker (click → popup), VolunteerPins (live from WebSocket), ResourceDesertAlert overlay
- Controls: category filter, hours_back slider (24h/72h/168h), geo_precision (6/7/8)
- API calls: GET /heatmap (Elasticsearch geo_point aggregation), GET /resource-deserts, GET /ward-stats

(app)/settings/weights — Urgency Weight Configuration

- Components: UrgencyWeightSliders (w1–w7), LiveScorePreview, CategoryBoostEditor (per-category multiplier map)
- Client-side constraint: additive weights (w1–w6) must sum ≤ 1.0 — enforced with zod.refine()
- Live preview: hypothetical urgency score recomputed locally using the scoring formula as sliders move — no API call until Save
- API calls: GET /weights, PUT /weights

field/* — Field Worker PWA

- Separate mobile-optimized shell: bottom tab navigation, camera integration for scan page
- Offline-capable: Background Sync queues POST /ingest/mobile submissions in IndexedDB, retries when online
- field/tasks/{id}: shows briefing_text in user's preferred_language, GPS check-in button (POST /tasks/{id}/start), outcome form (POST /tasks/{id}/complete)
- Pre-cached household data from last online session with 'Last updated X ago' stale indicator

SECTION 4 — API MAPPING & GAP ANALYSIS

4.1 Existing Backend API → Frontend View Mapping

Authentication & Session			
Frontend Action	Method	Endpoint	Notes
Tenant login	POST	/auth/login	Returns access + refresh tokens
Token refresh (background)	POST	/auth/refresh	Axios interceptor on 401
Logout	POST	/auth/logout	Revokes refresh token
Get current user	GET	/users/me	Populates auth store on mount
Create user (ngo_admin)	POST	/users	Settings → Users page
Create tenant	POST	/tenants	Platform admin panel only
Household Registry			
Frontend Action	Method	Endpoint	Notes
Household search	POST	/households/search	Full-text + geo + flag filters
Household profile	GET	/households/{id}?include_members=true	
Update household	PATCH	/households/{id}	Edit location, tier, landmarks
Identity resolution	POST	/households/resolve	Called as form location changes
Merge households	POST	/households/merge	Coordinator merge tool
Add consent	POST	/households/{id}/consent	Consent manager tab
Revoke consent	DELETE	/households/{id}/consent/{cid}	
Opt out household	POST	/households/{id}/opt-out	DPDP compliance
Household history	GET	/households/{id}/history	History timeline tab
Propose cross-tenant link	POST	/households/{id}/links	Settings → Links
Approve link	PATCH	/links/{id}/approve	Settings → Links
Data Ingestion			
Frontend Action	Method	Endpoint	Notes
Submit mobile form	POST	/ingest/mobile	MobileFormSubmission schema
Upload paper survey	POST	/ingest/image	Multipart, language_hints param
Submit WhatsApp/SMS text	POST	/ingest/text	source: whatsapp sms
Upload CSV	POST	/ingest/csv	Returns per-row results array
Review queue list	GET	/review?review_type=X	
Approve review	POST	/review/{id}/approve	With link_to_household
Reject review	POST	/review/{id}/reject	
List need records	GET	/needs?status=X&category;=Y	Priority queue source
Need record detail	GET	/needs/{id}	
Intelligence Engine			

Frontend Action	Method	Endpoint	Notes
Priority queue	GET	/priority-queue?limit=N&category;=X	
Escalation alerts	GET	/escalations	Dashboard alert panel
Geo-heatmap data	GET	/heatmap?category=X&hours;_back=N	ES geohash_grid
Ward stats	GET	/ward-stats	Tooltip on ward hover
Generate gap report	POST	/gap-report/generate	Kafka event emitted
Latest gap report	GET	/gap-report	Dashboard resource desert
Resource deserts	GET	/resource-deserts	Heatmap overlay layer
Score history	GET	/score-history/{need_id}	Urgency tooltip (7 terms)
Get urgency weights	GET	/weights	Settings → Weights
Update urgency weights	PUT	/weights	Settings → Weights
Full-text search	GET	/search?q=X&category;=Y	Global search bar (needs only)

Coordination Service

Frontend Action	Method	Endpoint	Notes
Register volunteer	POST	/volunteers	Settings → Volunteers
List volunteers	GET	/volunteers?active=true&skill;=X	
Volunteer profile	GET	/volunteers/{id}	
Update volunteer	PATCH	/volunteers/{id}	
GPS location ping	POST	/volunteers/{id}/location	Mobile, every 5 min
Volunteer burnout	GET	/volunteers/{id}/burnout	Profile page
Create task from need	POST	/tasks	Need detail → Create Task
List tasks	GET	/tasks?status=X	Kanban board
Get match candidates	GET	/tasks/{id}/matches	Dispatch panel
Dispatch volunteer	POST	/tasks/{id}/dispatch	Coordinator approves
Accept task	POST	/tasks/{id}/accept?volunteer_id=X	Volunteer app
Decline task	POST	/tasks/{id}/decline?volunteer_id=X	Volunteer app
Start task (check-in)	POST	/tasks/{id}/start?volunteer_id=X	Field app GPS
Complete task	POST	/tasks/{id}/complete?volunteer_id=X	Field outcome form
Close task	POST	/tasks/{id}/close	Coordinator post-review
Check dispatch timeouts	POST	/dispatch-timeouts/check	Scheduled call

Analytics Service

Frontend Action	Method	Endpoint	Notes
Impact metrics list	GET	/metrics/impact?period_type=monthly	Analytics dashboard
Compute period metrics	POST	/metrics/compute	Admin trigger
HII improvement table	GET	/households/improvement?window_days=30	
Single household HII	GET	/households/{id}/improvement	Profile tab
Volunteer performance	GET	/volunteers/performance	Analytics → Volunteers
Override summary	GET	/overrides/summary?days_back=30	

Frontend Action	Method	Endpoint	Notes
Override patterns	GET	/overrides/patterns?days_back=90	Where algo underperforms
Cross-NGO duplication	GET	/duplication-rate?days_back=30	Dashboard metric
Trigger feedback loop	POST	/feedback/process/{task_id}	Admin trigger
Funder report	GET	/funder-report?period_months=3	Funder portal

4.2 ■ Gap Analysis — Missing Backend Endpoints

The following 10 endpoints are required by the frontend based on the UI flows defined above but were NOT built in the Phase 0–5 backend implementation. All must be added before the corresponding frontend features can be developed.

■ GAP-01: Override Recording Endpoint

When a coordinator overrides the algorithm's suggestion, the dispatch endpoint currently does not record override context. `override_learning.py` has `record_override()` but no API exposes it. Must be called atomically with dispatch.

GAP-01 — PROPOSED ENDPOINT

```
POST /coordination/overrides/record
Authorization: Bearer {coordinator_token}
Request Body:
{ "task_id": "uuid", "suggested_vol_id": "uuid|null",
  "suggested_score": 0.78, "chosen_vol_id": "uuid",
  "override_reason": "string", "need_category": "food",
  "need_ward_id": "K-East", "need_urgency": 0.82 }
Response 201:
{ "override_id": "uuid", "recorded_at": "ISO8601" }
```

■ GAP-02: Unified Cross-Entity Search

The global search bar must search households, needs, and volunteers simultaneously. GET `/search` (intelligence service) only searches `need_records` via Elasticsearch. No cross-entity search exists.

GAP-02 — PROPOSED ENDPOINT

```
GET /search/global?q={query}&limit=5
Authorization: Bearer {token}
Response 200:
{ "query": "family near dargah",
  "results": {
    "needs": [ { need_id, category, urgency_score, _score } ],
    "households": [ { household_id, location_description, ward_id } ],
    "volunteers": [ { volunteer_id, skills, burnout_risk_score } ]
  }
}
```

■ GAP-03: Volunteer Availability Check

`availability_schedule` is stored as JSONB but no endpoint returns a parsed, date-specific availability view. Coordinators need to confirm a volunteer is actually available before dispatch, beyond the hard filter in the matching engine.

GAP-03 — PROPOSED ENDPOINT

```
GET /volunteers/{id}/availability?date=YYYY-MM-DD
Response 200:
{ "volunteer_id": "uuid", "date": "2025-04-23",
  "schedule": ["09:00-17:00"], "is_available_now": true,
  "active_task_count": 0, "burnout_risk_score": 0.31,
  "can_be_dispatched": true, "exclusion_reason": null }
```

■ GAP-04: Create Household from Review Queue

No single-step endpoint creates a household from a NeedRecord's NLP data, links the NeedRecord to the new household, and marks the review item approved atomically.

GAP-04 — PROPOSED ENDPOINT

```
POST /review/{review_id}/create-household
Request: { dwelling_type, economic_tier, landmark_tags,
  latitude, longitude, location_description, members[] }
Response 201:
{ "household_id": "uuid", "need_id": "uuid",
  "review_status": "approved",
  "household_resolution": "new_household_created" }
```

■ GAP-05: Notification Preferences & Test

The volunteer profile page needs to display current notification channel config and allow coordinators to send a test message to verify WhatsApp/SMS numbers. PATCH /volunteers/{id} accepts these fields but no GET reads just notification config.

GAP-05 — PROPOSED ENDPOINT

```
GET /volunteers/{id}/notifications
POST /volunteers/{id}/notifications/test
GET Response 200:
{ "preferred_channel": "whatsapp",
  "whatsapp_number": "+919876543210",
  "push_token_registered": true,
  "last_notification_status": "delivered" }
POST Request: { channel: 'whatsapp', message: 'Test' }
POST Response: { sent: true, external_id: 'SM...' }
```

■ GAP-06: Consent Bulk Status

ConsentManager must display status for all 5 consent types simultaneously. GET /households/{id} does not include consent records. Fetching separately per type would require 5 API calls.

GAP-06 — PROPOSED ENDPOINT

```
GET /households/{id}/consent
Response 200:
{ "opt_out": false, "consents": [
  { "consent_type": "data_collection",
    "is_active": true, "expires_at": "ISO8601",
    "collection_method": "digital_app",
    "language_used": "hi", "scope": { ... } },
  { "consent_type": "location_sharing", "is_active": false },
  { "consent_type": "cross_org_linking", "is_active": false }
] }
```

■ GAP-07: Task Briefing Preview

BriefingPreview component must show the coordinator the exact message that will be sent to the volunteer before dispatch is confirmed. Briefing generation currently happens server-side inside POST /tasks/{id}/dispatch with no preview step.

GAP-07 — PROPOSED ENDPOINT

```
POST /tasks/{id}/briefing/preview
Request: { volunteer_id: uuid, language: 'mr' }
Response 200:
{ "channel": "whatsapp", "language": "mr",
  "message": "■ NEXUS ■■■■■ ■■■■■\n...",
  "variables_used": {
    "category": "food", "location": "Ward K-East",
    "beneficiary_count": 5, "description": "..." } }
```

■ GAP-08: Ingestion Status Polling

PipelineResult component needs to poll processing status. In production, OCR + NLP is async (Kafka consumer). POST /ingest/image is synchronous in dev but will be async in production. Frontend needs a polling endpoint.

GAP-08 — PROPOSED ENDPOINT

```
GET /ingest/status/{raw_id}
Response 200:
{ "raw_id": "uuid",
  "status": "processing|processed|failed|duplicate",
  "need_id": "uuid|null",
  "household_status": "auto_linked|review_required|null",
  "nlp_confidence": 0.82,
  "is_duplicate": false,
  "routed_to_review": false,
  "processing_error": null }
```

■ GAP-09: Ward Lookup / Autocomplete

Household creation form, filter bars, and settings all need a searchable list of valid ward IDs with display names and current statistics. No endpoint returns this.

GAP-09 — PROPOSED ENDPOINT

```
GET /wards?q={search_term}&limit=20
Response 200:
{ "wards": [
  { "ward_id": "K-East",
    "display_name": "Kurla (East)",
    "centroid_lat": 19.0728, "centroid_lon": 72.8826,
    "open_needs_count": 34,
    "avg_urgency_score": 0.72,
    "is_resource_desert": false }
  ] }
Source: ward_stats table (fallback: DISTINCT ward_id from need_records)
```

■ GAP-10: Cross-Tenant Policy Configuration

Settings → Links page needs to read and update cross_tenant_linking_policy (auto_propose / coordinator_only / disabled) per tenant pair. No API exists for this.

GAP-10 — PROPOSED ENDPOINT

```
GET /settings/cross-tenant-policies
PATCH /settings/cross-tenant-policies/{partner_tenant_id}
GET Response 200:
{ "default_policy": "coordinator_only",
  "tenant_pairs": [
    { "partner_tenant_id": "uuid",
      "partner_name": "Pragati NGO Mumbai",
      "policy": "coordinator_only",
      "linked_households": 12 }
  ] }
PATCH Request: { policy: 'auto_propose' }
```


SECTION 5 — STATE MANAGEMENT STRATEGY

5.1 Architecture Overview

State is divided into three tiers based on scope and lifecycle. Global state (Zustand) holds auth tokens, tenant context, and the shared WebSocket connection. Server state (React Query) handles all API data with configured cache TTLs, background refetch, and optimistic updates. Local component state (useState) handles ephemeral UI interactions that don't need to survive a navigation.

5.2 Zustand Global Store Slices

authStore

Field	Type	Purpose
accessToken	string null	JWT in memory ONLY — never localStorage or sessionStorage
userId	string null	Decoded from JWT sub claim
tenantId	string null	Decoded from JWT tenant_id claim
role	UserRole	field_worker coordinator ngo_admin funder_readonly platform_admin
expiresAt	number null	Unix timestamp — drives Axios interceptor refresh logic
isAuthenticated	boolean	Derived: accessToken && expiresAt > Date.now()

tenantStore

Field	Type	Purpose
tenantId	string null	Injected into every Axios request as X-Tenant-ID header
tenantSlug	string null	URL routing and display
tenantName	string null	UI display
settings	TenantSettings	cross_tenant_linking_policy, urgency weight config

socketStore

Field	Type	Purpose
socket	Socket null	Single shared Socket.io instance initialized in (app)/layout.tsx
connected	boolean	Connection status indicator in topbar
lastEvent	NexusEvent null	Most recent event for LiveEventFeed

notificationStore

Field	Type	Purpose
toasts	Toast[]	Max 5 visible WebSocket-triggered toasts
escalationCount	number	Unread badge on sidebar nav Escalations link
lastGapReport	GapReportSummary null	Drives ResourceDesertAlert banner

mapStore

Field	Type	Purpose
viewport	MapViewState	lat/lon/zoom — shared by all map layer components

Field	Type	Purpose
categoryFilter	string null	Active heatmap category filter
hourFilter	number	hours_back param (24/72/168)
selectedNeed	string null	Need ID for popup display

5.3 React Query Cache & Invalidation Strategy

All query keys include tenantId to enforce the same isolation as the backend RLS policies. staleTime is tuned per data volatility — real-time data relies on WebSocket invalidation, heavy aggregations are cached aggressively.

Data Type	staleTime	gcTime	Justification
Priority queue	30s	2 min	Real-time; WebSocket triggers invalidation
Urgency scores	60s	5 min	Recomputed on schedule + events
Household profile	5 min	30 min	Rarely changes mid-session
Household history	2 min	10 min	New events via WebSocket
Match candidates	30s	2 min	Burnout/availability changes fast
Ward stats	5 min	30 min	Scheduler refreshes every 4h
Impact metrics	10 min	1 hr	Monthly aggregations
Funder report	30 min	2 hr	Heavy query, rarely changes
Heatmap data	1 min	5 min	ES geo-aggregation is expensive
Urgency weights	10 min	1 hr	Only changes on coordinator action
Gap report	5 min	30 min	Generated by 4h scheduler
Volunteer burnout	2 min	10 min	Updated after every task completion

5.4 WebSocket Event → React Query Invalidation Map

WebSocket Event	Query Key(s) Invalidated	Strategy
need.created	queryKeys.priorityQueue(tenantId, *)	Prepend item + invalidate
need.urgency_updated	queryKeys.need(tenantId, needId)	Partial cache update
task.dispatched	queryKeys.tasks(tenantId), queryKeys.task(tenantId, id)	Invalidate
task.accepted	queryKeys.task(tenantId, id)	Optimistic already applied
task.completed	queryKeys.tasks, queryKeys.household(tenantId, hhId)	Invalidate + feedback
volunteer.location_updated	mapStore.setVolunteerLocation()	Direct store update, no query
household.merged	queryKeys.households(tenantId)	Invalidate household list
household.consent_updated	queryKeys.hhConsent(tenantId, hhId)	Invalidate consent
analytics.gap_report.generated	queryKeys.gapReport(tenantId) + notificationStore	Both
need.status_changed	queryKeys.need(tenantId, id) + heatmap invalidate	

5.5 Axios Interceptor Architecture

The Axios instance in lib/api/client.ts implements three interceptors that bind the frontend to the backend security model:

AXIOS INTERCEPTORS

REQUEST INTERCEPTORS:

1. Inject: Authorization: Bearer {accessToken from authStore}

2. Inject: X-Tenant-ID: {tenantId from tenantStore}

3. Inject: X-Correlation-ID: {uuid()} for distributed tracing

4. Inject: Accept-Language: {locale} for i18n hint

RESPONSE INTERCEPTORS:

1. HTTP 401 → call POST /auth/refresh → retry original request with new token
→ if refresh also fails → clearAuth() + redirect to /login

2. HTTP 403 → redirect to /unauthorized with permission context

3. HTTP 429 → exponential backoff with jitter (max 3 retries, base 1s)

5.6 Local Component State (Not in Zustand or React Query)

Component	Local State Managed
MobileNeedForm	react-hook-form state, submission status, GPS permission state
ImageUploadZone	Dropped file blob, upload progress %, compression ratio, OCR text edits
CSVImporter	Parsed rows (PapaParse), column mapping object, per-row validation errors
HouseholdSideBySide (Merge)	Which household is 'primary', field-level selection per row
UrgencyWeightSliders	Slider values (local preview before PUT /weights — no API call)
DispatchControls	Selected volunteer ID before confirming, override reason text input
ConsentManager	Expanded/collapsed state per consent type accordion
HistoryTimeline	Active event_type filter (local, not persisted to URL)
PriorityQueueTable	Sort column, sort direction, selected row IDs for bulk actions
BriefingPreview	Preview text (from GAP-07 endpoint), language selector value

5.7 URL State (nuqs) — Filter States Persisted to Search Params

Page	URL Parameters (nuqs library)
/needs	?category=food&ward;=K-East&vuln;=has_child&page;=2
/households	?q=dargah&ward;=K-East&min;_vuln=0.5&status;=active
/intelligence/heatmap	?category=health&hours;=168&precision;=7
/analytics	?period_type=monthly._start=2025-01-01
/volunteers	?skill=medical&active;=true&ward;=K-East
/tasks	?status=dispatched&page;=1

5.8 Offline Strategy — Field Worker PWA

The field/* route group targets field workers in low-connectivity rural areas where 60–70% of deployments may lack reliable internet. Four mechanisms combine to ensure zero data loss:

- App Shell Cache: next-pwa generates a service worker that caches the entire React bundle and static assets on first load. Subsequent visits load instantly regardless of connectivity.
- Background Sync for form submissions: POST /ingest/mobile failures queue the FormData in IndexedDB via the Background Sync API. The service worker retries automatically when online. The user sees a 'Queued — will sync when connected' status badge.
- Pre-cached household data: When a field worker navigates to a household page while online, the fetch response is stored in the service worker Cache Storage. Offline access shows the cached profile with a 'Last updated X ago'

Confidential — Internal Architecture Document

Page 19

- banner and disabled mutation buttons.
- Graceful degradation: WebSocket connection, Mapbox heatmap, and real-time features automatically disable when `navigator.onLine` is false. A persistent banner shows connectivity status. No error states — just silently degraded functionality.

End of NEXUS Frontend Architecture & Requirements Document v1.0. This document was generated from the Phase 0–5 backend implementation and reflects all 70+ API endpoints, 5 user roles, 6 microservices, and 10 identified frontend gaps requiring backend additions before development begins.